

Sistemas Operacionais

Jantar dos Filósofos

Professor Romário da Silva Borges

Problemas clássicos de sincronização

A área de sistemas operacionais possui **problemas clássicos de sincronização**.

Situações recorrentes

Representam cenários que aparecem repetidamente em sistemas reais.

Teste de soluções

Ajudam a avaliar a eficácia de diferentes primitivas de sincronização.

Base prática

Servem como referência para problemas do mundo real.

Por que estudar problemas clássicos?

Problemas clássicos permitem analisar:

→ **Disputa por recursos**

→ **Exclusão mútua**

→ **Deadlock e Starvation**

→ **Eficiência e corretude dos algoritmos concorrentes**

O Problema dos Produtores e Consumidores

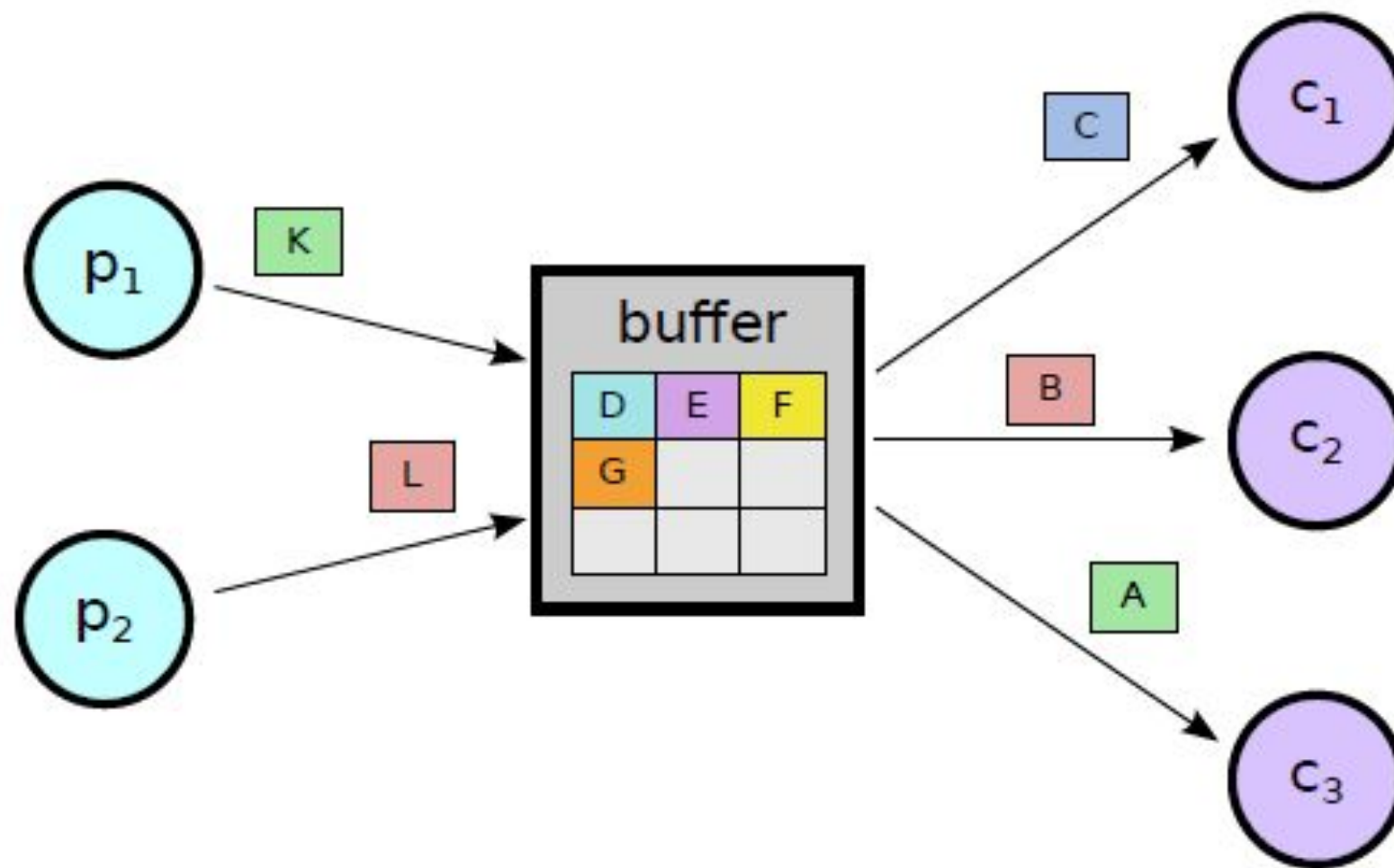


Figura 12.1: O problema dos produtores/consumidores.

Sobre o Problema:

Este problema também é conhecido como o problema do buffer limitado, e consiste em coordenar o acesso de tarefas (processos ou threads) a um buffer compartilhado com capacidade de armazenamento limitada a N itens (que podem ser inteiros, registros, mensagens, etc.).

Solução

A solução do problema dos produtores/consumidores envolve três aspectos de coordenação distintos e complementares:

- A exclusão mútua no acesso ao buffer é necessária para impedir condições de disputa entre produtores e consumidores, evitando que o conteúdo armazenado seja corrompido.
- Quando o buffer estiver cheio, os produtores devem ser suspensos e aguardar até que novos espaços sejam liberados.
- Quando o buffer estiver vazio, os consumidores devem ser suspensos e esperar até que novos itens sejam adicionados para consumo.



- Produtor espera vaga
- Consumidor espera item
- Mutex protege o buffer

```
// produtor  
wait(vazio); wait(mutex);  
put(item);  
signal(mutex); signal(cheio);
```

```
// consumidor  
wait(cheio); wait(mutex);  
item = get();  
signal(mutex); signal(vazio);
```

Leitores e escritores

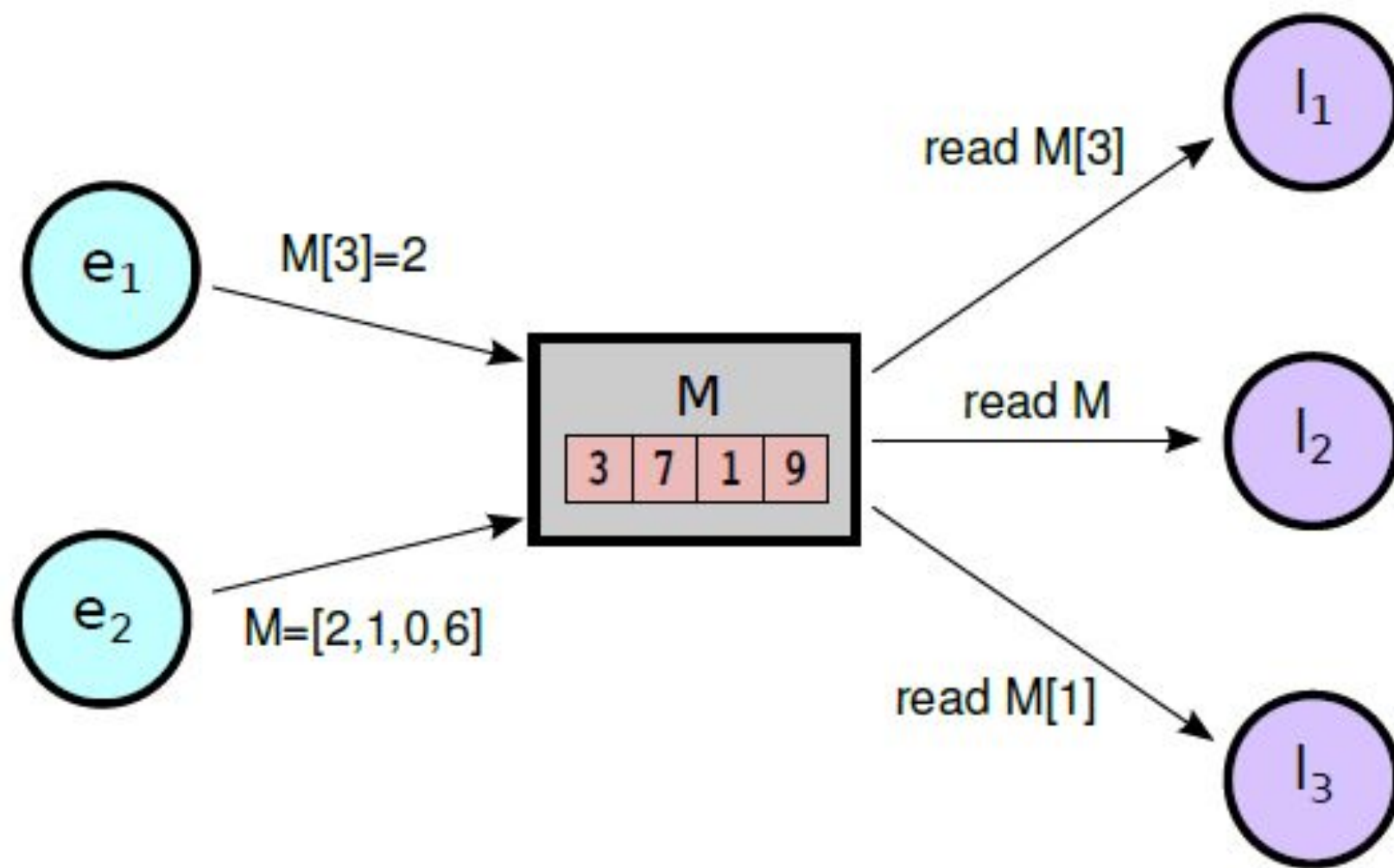


Figura 12.2: O problema dos leitores/escritores.

Sobre o Problema:

Nesse problema, várias tarefas acessam simultaneamente uma área de memória compartilhada, podendo realizar operações de leitura ou escrita. Conforme as condições de Bernstein, as operações de leitura podem ocorrer em paralelo, já que não interferem entre si. Por outro lado, as operações de escrita precisam ter acesso exclusivo à memória compartilhada, evitando condições de disputa e possíveis inconsistências nos dados.

Solução

Uma solução simples para esse problema seria utilizar um mutex ou um semáforo inicializado com valor 1 para controlar o acesso à área compartilhada. Dessa maneira, apenas um processo poderia acessá-la por vez, garantindo a integridade das operações realizadas.

Entretanto, essa abordagem apresenta baixo desempenho, pois limita desnecessariamente o acesso dos leitores. Como as operações de leitura não modificam os dados armazenados, vários leitores poderiam acessar a área compartilhada simultaneamente sem causar conflitos ou comprometer a integridade das informações.

E se houvesse uma solução de priorização para os leitores?

O jantar dos selvagens



```
1 task cozinheiro ()
2 {
3     while (1)
4     {
5         encher_caldeirao () ;
6         dormir () ;
7     }
8 }
9
10 task selvagem ()
11 {
12     while (1)
13     {
14         servir () ;
15         comer () ;
16     }
17 }
```

Sobre o Problema:

O problema do Jantar dos Selvagens é uma variação do problema dos produtores e consumidores. Nele, uma tribo possui um caldeirão compartilhado com capacidade para N porções de comida.

Quando um selvagem deseja comer, ele retira uma porção do caldeirão. Caso o caldeirão esteja vazio, ele acorda o cozinheiro e espera. O cozinheiro então prepara novas porções, enche novamente o caldeirão e volta a dormir. Depois disso, os selvagens podem continuar se servindo.

Sobre o Problema

As principais regras de sincronização desse problema são:

- Apenas um selvagem pode se servir do caldeirão por vez, embora vários possam comer ao mesmo tempo.
- Nenhum selvagem pode se servir quando o caldeirão estiver vazio.
- O cozinheiro só pode colocar novas porções no caldeirão quando ele estiver completamente vazio.

Solução?

```
1  int porcoes = 0 ;
2  mutex mcald ;
3  semaphore cald_vazio ;
4  semaphore cald_cheio ;
5
6  task cozinheiro ()
7  {
8      while (1)
9      {
10
11
12
13      }
14  }
```

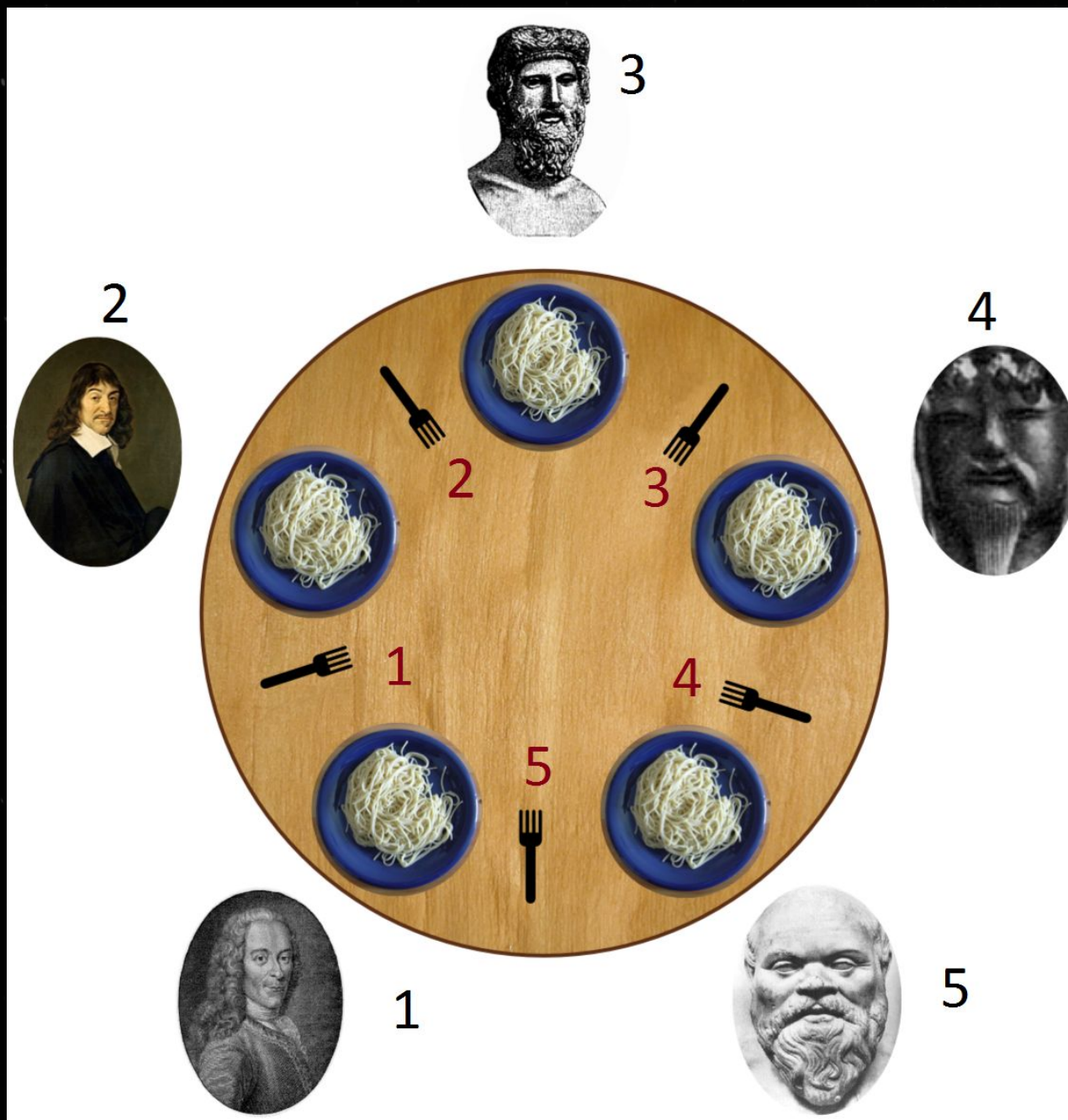
```
task selvagem ()
{
    while (1)
    {

    }
}
```

Solução

Uma solução simples para esse problema, apresentada em [Downey, 2016], é utilizar semáforos para controlar o acesso ao caldeirão e coordenar a comunicação entre os selvagens e o cozinheiro. Um semáforo garante que apenas um selvagem por vez acesse o caldeirão, enquanto outros semáforos são usados para acordar o cozinheiro quando o caldeirão fica vazio e para fazer os selvagens aguardarem até que ele seja novamente abastecido.

O jantar dos filósofos



O que o problema ensina?

Recursos limitados

Processos independentes

Exclusão mútua

Risco de deadlock

Risco de starvation

Necessidade de algoritmos corretos

Sobre o Problema:

- O jantar dos filósofos é um dos problemas clássicos de sincronização em sistemas concorrentes. Nele, cinco filósofos ficam sentados ao redor de uma mesa e alternam entre pensar e comer.
- Na mesa existem cinco pratos e cinco garfos compartilhados, além de um prato de macarrão no centro. Para conseguir comer, cada filósofo precisa pegar dois garfos: o que está à sua direita e o que está à sua esquerda. Como os garfos são compartilhados, dois filósofos vizinhos não podem comer ao mesmo tempo.
- Os filósofos não se comunicam entre si e também não conseguem saber diretamente o estado dos demais. E não deve ter um mecanismo centralizado de controle. O objetivo do problema é criar uma forma de coordenar o uso dos garfos para que todos consigam alternar entre pensar e comer sem conflitos, bloqueios ou uso incorreto dos recursos compartilhados.

```
1 #define NUMFILO 5
2 semaphore hashi [NUMFILO] ; // um semáforo para cada palito (iniciam em 1)
3
4 task filosofo (int i)          // filósofo i (entre 0 e 4)
5 {
6     int dir = i ;
7     int esq = (i+1) % NUMFILO ;
8
9     while (1)
10    {
11        meditar () ;
12        down (hashi [dir]) ;      // pega palito direito
13        down (hashi [esq]) ;      // pega palito esquerdo
14        comer () ;
15        up (hashi [dir]) ;         // devolve palito direito
16        up (hashi [esq]) ;         // devolve palito esquerdo
17    }
18 }
```

Solução?

```
1 #define NUMFILO 5
2 semaphore hashi [NUMFILO] ; //
3 semaphore saleiro ; //
4
5 task filosofo (int i)
6 {
7     int dir = i ;
8     int esq = (i+1) % NUMFILO] ;
9
10    while (1)
11    {
12
13
14
15
16
17
18
19
20    }
21 }
```

```

1  #define NUMFILO 5
2  semaphore hashi [NUMFILO] ; // um semáforo para cada palito (iniciam em 1)
3  semaphore saleiro ;          // um semáforo para o saleiro
4
5  task filosofo (int i)          // filósofo i (entre 0 e 4)
6  {
7      int dir = i ;
8      int esq = (i+1) % NUMFILO ;
9
10     while (1)
11     {
12         meditar () ;
13         down (saleiro) ;          // pega saleiro
14         down (hashi [dir]) ;      // pega palito direito
15         down (hashi [esq]) ;      // pega palito esquerdo
16         up (saleiro) ;            // devolve saleiro
17         comer () ;
18         up (hashi [dir]) ;        // devolve palito direito
19         up (hashi [esq]) ;        // devolve palito esquerdo
20     }
21 }

```

Solução

Uma solução simples para o problema do jantar dos filósofos é utilizar um “saleiro” como forma de controle. Quando um filósofo quiser comer, ele deve primeiro pegar o saleiro e, em seguida, pegar os dois garfos necessários. Depois de conseguir os dois garfos, ele devolve o saleiro à mesa e começa a comer.

Dessa forma, apenas um filósofo por vez pode tentar pegar os garfos, evitando conflitos e situações de bloqueio entre eles.

Qual o ponto fraco dessa solução?

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

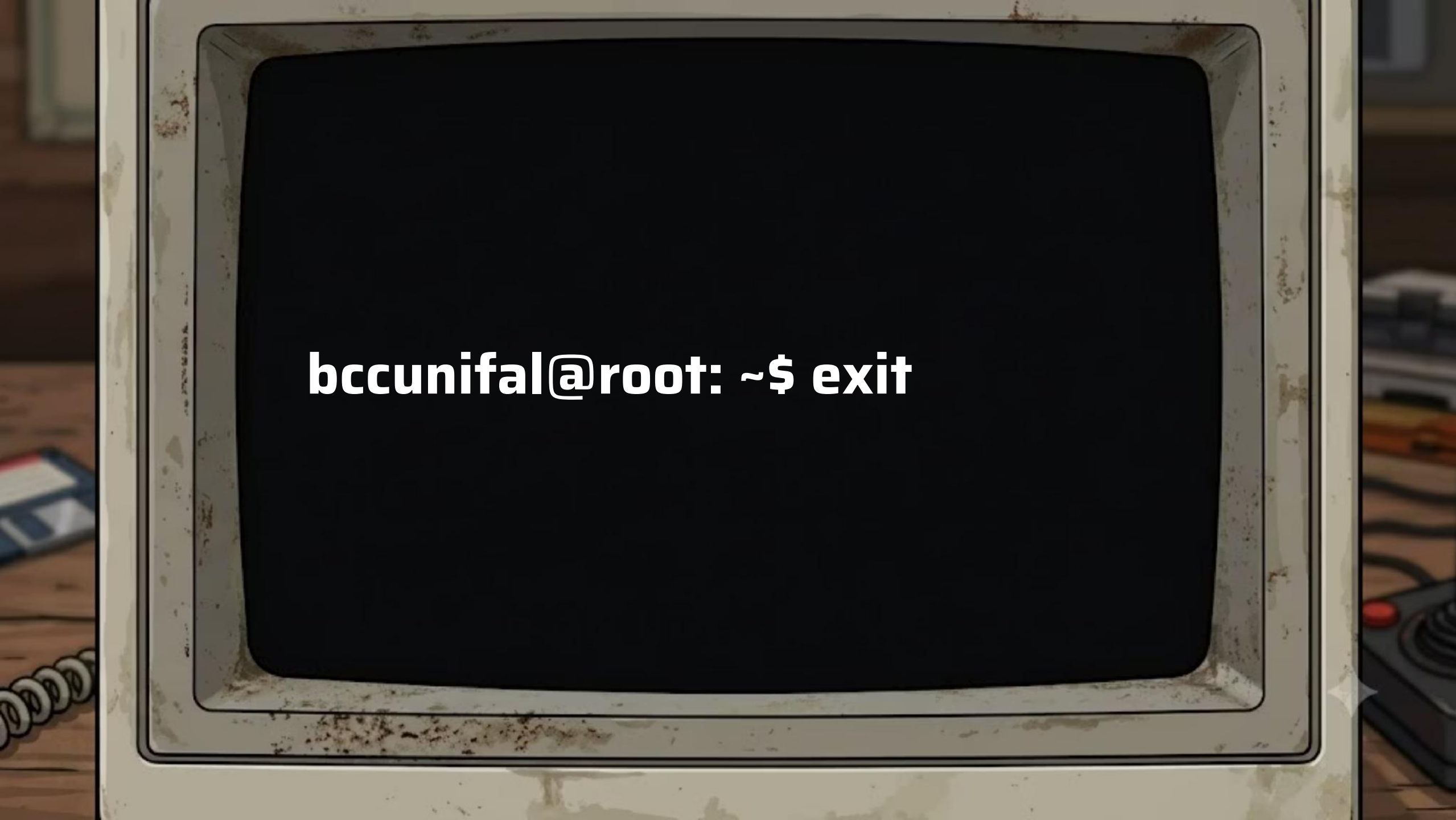
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit